অধ্যায় ১১

Error Handling, Debugging এবং Defensive Programming

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- Syntax, Runtime ও Logical Error-এর পার্থক্য করতে, এবং JavaScript-এর Built-in Error-এর ধরন চিনতে।
- `throw`, `try`, `catch` ও `finally` দিয়ে ত্রুটি তৈরি, ধরা ও পরিষ্কারের কাজ নিশ্চিত করতে।
- নিজের Error তৈরি করতে, এবং `cause` দিয়ে মূল কারণ ধরে রাখতে।
- প্রত্যাশিত (ব্যবহারকারী ঠিক করতে পারেন) ও অপ্রত্যাশিত (Program-এর ভুল) ত্রুটি আলাদাভাবে সামলাতে।
- Trust Boundary-তে বাইরের তথ্য যাচাই করে Defensive Program লিখতে।
- Asynchronous ত্রুটি এবং Global Error Handler ব্যবহার করতে।
- Stack Trace পড়তে, এবং DevTools-এর Breakpoint দিয়ে Bug খুঁজতে।
- ত্রুটি সামলানোর প্রচলিত ভুলগুলো (Anti-pattern) চিনে এড়াতে।
- একটি সম্পূর্ণ **OrderGuard: Order Validation** Web Application তৈরি ও পরীক্ষা করতে।

পূর্বশর্ত

- অধ্যায় ৮: Regular Expression দিয়ে সরল যাচাই।
- অধ্যায় ৯: Form, `submit` Event এবং Accessibility-র অভ্যাস।
- অধ্যায় ১০: `async` / `await`, Promise-এর rejection এবং কৃত্রিম Server দিয়ে Test।
- `chapter-11` নামে একটি নতুন Folder।

এই অধ্যায় কেন গুরুত্বপূর্ণ

প্রতিটি বাস্তব Program-এ কিছু না কিছু ভুল হবে। ব্যবহারকারী Email-এর ঘরে নাম লিখবেন, Network হঠাৎ বিচ্ছিন্ন হবে, Server অপ্রত্যাশিত তথ্য পাঠাবে, আর আপনার নিজের Code-এ একটি Bug লুকিয়ে থাকবে। প্রশ্ন এটি নয় যে ত্রুটি ঘটবে কি না, প্রশ্ন হলো ঘটলে কী হবে।

একটি দুর্বল Program তখন সাদা পর্দা দেখায়, অথবা আরও খারাপ: চুপচাপ ভুল তথ্য সংরক্ষণ করে। একটি নির্ভরযোগ্য Program ব্যবহারকারীকে জানায় কী ঠিক করতে হবে, যা ঠিক করা সম্ভব নয় তা নিরাপদে থামায়, এবং ডেভেলপারের জন্য যথেষ্ট তথ্য রেখে যায় যাতে Bug খুঁজে পাওয়া যায়। এই অধ্যায়ে আপনি শিখবেন কোন ত্রুটি ধরবেন, কোনটি ধরবেন না, এবং ধরার পর কী করবেন।

১১.১ তিন ধরনের ত্রুটি

Syntax Error ঘটে যখন Code JavaScript-এর ব্যাকরণ মানে না। Program চলা শুরুই করে না।

`syntax.js` · JavaScript — চলবে: Node.js

```
console.log('এই লাইনও চলবে না');
const = 42;
```

Error Message

```
SyntaxError: Unexpected token '='
```

প্রথম লাইনটিও চলেনি, কারণ JavaScript পুরো File আগে পড়ে নেয়, এবং ব্যাকরণের ভুল পেলে কিছুই চালায় না।

Runtime Error ঘটে Program চলার সময়, যখন একটি কাজ করা সম্ভব হয় না।

`runtime.js` · JavaScript — চলবে: Node.js

```
const user = null;

console.log('এই লাইন চলে');
console.log(user.name);
console.log('এই লাইন চলে না');
```

Error Message

```
এই লাইন চলে
TypeError: Cannot read properties of null (reading 'name')
```

Logical Error সবচেয়ে বিপজ্জনক: Program চলে, কোনো বার্তা দেয় না, কিন্তু ফল ভুল।

logical.js · JavaScript চলবে: Browser Console ও Node.js

```
function priceAfterDiscount(price, percent) {
  return price - percent;
}

console.log(priceAfterDiscount(100, 10));
console.log(priceAfterDiscount(2000, 10));
```

Output

```
90
1990
```

১০০ টাকায় ফল ঘটনাক্রমে ঠিক (১০%-এ ৯০), কিন্তু ২০০০ টাকায় ১০% ছাড়ের পর ১৮০০ হওয়ার কথা। সূত্রে `price * (1 - percent / 100)` লেখা উচিত ছিল। এমন ভুল কোনো Error-এর বার্তা দেয় না; এগুলো ধরে Automated Test, যেখানে একাধিক মান দিয়ে প্রত্যাশিত ফল মেলানো হয়।

১১.২ Error Object

Runtime-এ কিছু ভুল হলে JavaScript একটি **Error Object** তৈরি করে **ছোড়ে** (throw)। কেউ সেটি না ধরলে Program থামে। প্রতিটি Error Object-এ তিনটি প্রধান Property:

Property	কী থাকে	কার জন্য
`name`	ধরন: `TypeError`, `RangeError` ইত্যাদি	Code, ধরন অনুযায়ী সিদ্ধান্তের জন্য
`message`	কী ভুল হয়েছে তার বিবরণ	ডেভেলপার
`stack`	কোন Function-এর কোন লাইনে ঘটেছে	ডেভেলপার, Debugging-এর জন্য

error-types.js · JavaScript চলবে: Browser Console ও Node.js

```
const attempts = [
  () => null.length,
  () => notDeclared + 1,
  () => (12.5).toFixed(500),
  () => JSON.parse('{bad'),
  () => decodeURIComponent('%'),
];

for (const attempt of attempts) {
  try {
    attempt();
  } catch (error) {
    console.log(`${error.name}, Error-ও? ${error instanceof Error}`);
  }
}
```

Output

```
TypeError, Error-ও? true
ReferenceError, Error-ও? true
RangeError, Error-ও? true
SyntaxError, Error-ও? true
URIError, Error-ও? true
```

ধরন	কখন
`TypeError`	ভুল ধরনের মানের ওপর কাজ: `null`-এর Property, Function নয় এমন কিছু Call
`ReferenceError`	ঘোষণা করা হয়নি এমন Variable
`RangeError`	গ্রহণযোগ্য সীমার বাইরের মান
`SyntaxError`	Code-এর ব্যাকরণ ভুল, অথবা `JSON.parse`-এ ভাঙা JSON
`URIError`	ভুল গঠনের URL-অংশ Decode করা

সবগুলোই `Error` থেকে তৈরি, তাই `instanceof Error` সবার জন্য `true`। `message`-এর লেখা Browser ও Node.js-এর সংস্করণভেদে বদলাতে পারে; Code-এ সিদ্ধান্ত নেওয়ার জন্য `message`-এর লেখা নয়, `name` বা `instanceof` ব্যবহার করুন।

১১.৩ throw, try, catch এবং finally

নিজের Code-এ ভুল ধরা পড়লে আপনিও `throw` করতে পারেন। সব সময় একটি Error Object ছুড়ুন, সাধারণ লেখা নয়: Error Object-এ `stack` থাকে, লেখায় থাকে না।

throw.js · JavaScript চলবে: Browser Console ও Node.js

```
function readPrice(item) {
  if (typeof item?.price !== 'number') {
    throw new TypeError('পণ্যের দাম একটি সংখ্যা হতে হবে');
  }
  if (item.price < 0) {
    throw new RangeError(`দাম ঋণাত্মক হতে পারে না: ${item.price}`);
  }
  return item.price;
}

for (const item of [{ price: 250 }, { price: '250' }, { price: -5 }]) {
  try {
    console.log(`দাম: ${readPrice(item)}`);
  } catch (error) {
    console.log(`${error.name}: ${error.message}`);
  }
}
```

Output

```
দাম: 250
TypeError: পণ্যের দাম একটি সংখ্যা হতে হবে
RangeError: দাম ঋণাত্মক হতে পারে না: -5
```

`throw` হওয়ার সঙ্গে সঙ্গে Function থামে, এবং নিয়ন্ত্রণ চলে যায় সবচেয়ে কাছের `catch`-এ, প্রয়োজনে একাধিক Function পেরিয়ে। একটি ত্রুটিতে পুরো Loop থামেনি: প্রতিটি পণ্য আলাদা `try`-এর ভেতরে।

finally সব সময় চলে

finally.js · JavaScript চলবে: Browser Console ও Node.js

```
function runStep(input) {
  try {
    console.log('১. চেষ্টা');
    if (input < 0) {
      throw new RangeError('ঋণাত্মক');
    }
    return 'সফল';
  } catch (error) {
    console.log(`২. ধরা পড়েছে: ${error.message}`);
    return 'ব্যর্থ';
  } finally {
    console.log('৩. finally সব সময়');
  }
}
```

```
console.log(runStep(5));
console.log(runStep(-1));
```

Output

```
১. চেষ্টা
৩. finally সব সময়
সফল
১. চেষ্টা
২. ধরা পড়েছে: ঋণাত্মক
৩. finally সব সময়
ব্যর্থ
```

`try`-এর ভেতরে `return` থাকলেও Function ফেরত যাওয়ার আগে `finally` চলেছে। তাই পরিষ্কারের কাজ—Loading বন্ধ, Button আবার চালু, খোলা File বন্ধ—`finally`-তে রাখুন: সফল, ব্যর্থ বা আগেভাগে ফেরত, সব পথেই তা ঘটবে।

> **মনে রাখুন:** `catch` শুধু তার নিজের `try`-এর ভেতরের ত্রুটি ধরে, এবং শুধু সেই মুহূর্তে চলমান Code-এর। একটি `setTimeout`-এর Callback পরে চলে, তাই বাইরের `try` তার ত্রুটি ধরতে পারে না। Promise-এর ত্রুটি ধরতে `await` অপরিহার্য (অধ্যায় ১০)।

১১.৪ নিজের Error তৈরি

Built-in ধরনগুলো বলে কী ধরনের ভুল, কিন্তু আপনার Program-এর ভাষায় নয়। "ব্যবহারকারী ভুল তথ্য দিয়েছেন" এবং "Program-এ Bug আছে"—দুটিই `TypeError` হতে পারে, অথচ সামলাতে হয় সম্পূর্ণ আলাদাভাবে। নিজের Error-এর ধরন তৈরি করলে এই পার্থক্য স্পষ্ট হয়।

custom-error.js · JavaScript — চলবে: Browser Console ও Node.js

```
class ValidationError extends Error {
  constructor(field, message) {
    super(message);
    this.name = 'ValidationError';
    this.field = field;
  }
}

function parseAge(text) {
  const age = Number(text);

  if (!Number.isInteger(age) || age < 0 || age > 150) {
    throw new ValidationError('age', `বয়স সঠিক নয়: "${text}"`);
  }
  return age;
```

```
}

for (const input of ['32', 'abc', '-4']) {
  try {
    console.log(`বয়স: ${parseAge(input)}`);
  } catch (error) {
    if (!(error instanceof ValidationError)) {
      throw error;
    }
    console.log(`${error.field} ঘরে: ${error.message}`);
  }
}
```

Output

```
বয়স: 32
age ঘরে: বয়স সঠিক নয়: "abc"
age ঘরে: বয়স সঠিক নয়: "-4"
```

- `class ValidationError extends Error` `Error`-এর সব বৈশিষ্ট্যসহ একটি নতুন ধরন তৈরি করে। `super(message)` মূল `Error`-কে বার্তা দেয়; তারপর নিজের Property যোগ হয়। Class-এর পূর্ণ আলোচনা অধ্যায় ১৩-এ; এখানে এই ছকটুকুই যথেষ্ট।
- `this.name` বসানো জরুরি; নইলে Stack ও Console-এ শুধু `Error` দেখাবে।
- **`catch`-এ ধরন যাচাই, এবং অচেনা হলে আবার `throw`।** এই `catch` শুধু `ValidationError` সামলাতে জানে। অন্য কিছু এলে (যেমন একটি Bug-এর `TypeError`) তা চাপা না দিয়ে আবার ছুড়ে দেওয়া হয়েছে, যাতে ওপরের স্তর বা Console তা দেখতে পায়। এটি এই অধ্যায়ের সবচেয়ে গুরুত্বপূর্ণ অভ্যাস।

cause: মূল কারণ ধরে রাখা

নিচের স্তরের একটি ত্রুটিকে ওপরের স্তরের ভাষায় নতুন করে প্রকাশ করার সময় মূল ত্রুটি হারানো উচিত নয়।

cause.js · JavaScript চলবে: Browser Console ও Node.js

```
function loadSettings(text) {
  try {
    return JSON.parse(text);
  } catch (error) {
    throw new Error('Settings পড়া যায়নি', { cause: error });
  }
}

try {
  loadSettings('{oops');
} catch (error) {
  console.log(error.message);
  console.log(`মূল কারণ: ${error.cause.name}`);
}
```

Output

```
Settings পড়া যায়নি
মূল কারণ: SyntaxError
```

ওপরের স্তর পেল বোধগম্য বার্তা, আর Debugging-এর জন্য `cause`-এ রইল আসল `SyntaxError`, তার নিজের `stack`-সহ। Browser ও Node.js-এর Console দুটিই `cause` দেখায়।

১১.৫ প্রত্যাশিত বনাম অপ্রত্যাশিত ত্রুটি

ত্রুটি সামলানোর সবচেয়ে গুরুত্বপূর্ণ সিদ্ধান্ত হলো ত্রুটিটি কোন শ্রেণির।

	প্রত্যাশিত ত্রুটি	অপ্রত্যাশিত ত্রুটি
উদাহরণ	ভুল Email, শূন্য পরিমাণ, স্টক শেষ, Network বিচ্ছিন্ন	`undefined`-এর Property পড়া, নামের বানান ভুল, ভুল হিসাব
কার দায়	পরিবেশ বা ব্যবহারকারী	Program-এর Code
কে ঠিক করবেন	ব্যবহারকারী, বা কিছুক্ষণ পর আপনা থেকে	ডেভেলপার
ব্যবহারকারী দেখবেন	নির্দিষ্ট, কী করতে হবে সহ বার্তা	সাধারণ বার্তা: "অপ্রত্যাশিত সমস্যা"
ডেভেলপার পাবেন	প্রয়োজনে সংক্ষিপ্ত Log	পুরো Error ও Stack, `console.error` বা Error Reporting-এ
Program	স্বাভাবিকভাবে চলে	নিরাপদে থামে; ভুল অবস্থায় এগোয় না

প্রত্যাশিত ত্রুটি ধরুন যত কাছে সম্ভব, যেখানে আপনি জানেন কী করতে হবে। অপ্রত্যাশিত ত্রুটি ধরবেন না; সেগুলো একেবারে ওপরের স্তরে একটি জায়গায় পৌঁছাতে দিন, যেখানে সাধারণ বার্তা দেখানো এবং পুরো বিবরণ Log করা হয়।

সবচেয়ে প্রচলিত ভুল হলো দুটিকে একসঙ্গে ধরে ফেলা:

swallowed.js · JavaScript — চলবে: Browser Console ও Node.js

```
const DISCOUNT_RATES = { EID10: 10, NEW5: 5 };

// ভুল: সব ত্রুটি চাপা পড়ে, এমনকি Code-এর বানান ভুলও।
function discountFor(code) {
  try {
    return DISCOUNT_RATE[code.toUpperCase()] ?? 0;
  } catch {
    return 0;
  }
}

// সঠিক: প্রত্যাশিত ক্ষেত্র নিজে যাচাই; বাকি সব ত্রুটি দৃশ্যমান থাকে।
function discountForFixed(code) {
  if (typeof code !== 'string') {
    return 0;
  }
  return DISCOUNT_RATES[code.toUpperCase()] ?? 0;
}

console.log(discountFor('eid10'));
console.log(discountForFixed('eid10'));
console.log(discountForFixed(undefined));
```

Output

```
0
10
0
```

প্রথম Function-এ `DISCOUNT_RATES`-এর বানান ভুল (`DISCOUNT_RATE`)। এটি একটি `ReferenceError`, কিন্তু খালি `catch` তা গিলে ফেলেছে, এবং প্রতিটি গ্রাহক চুপচাপ ছাড় হারাচ্ছেন। Test ছাড়া এই Bug মাসের পর মাস টিকে থাকতে পারে। লেখকের উদ্দেশ্য ছিল "`code` ভুল হলে ০", এবং দ্বিতীয় Function ঠিক সেটিই সরাসরি যাচাই করেছে, `try` ছাড়া।

১১.৬ Defensive Programming: Trust Boundary

Trust Boundary হলো সেই সীমানা যেখানে বাইরের তথ্য আপনার Program-এ ঢোকে: Form, URL, `localStorage`, Server-এর সাড়া, অন্য Team-এর Library। সীমানার বাইরের তথ্যকে বিশ্বাস করবেন না; সীমানায় একবার ভালোভাবে যাচাই করুন, তারপর ভেতরের Code পরিষ্কার তথ্য নিয়ে নিশ্চিন্তে কাজ করুক।

সংখ্যা রূপান্তরের কয়েকটি ফাঁদ:

number-traps.js · JavaScript চলবে: Browser Console ও Node.js

```
console.log(Number(''), Number('  '), Number('1e3'), Number('0x10'));
console.log(Number('12abc'), parseInt('12abc', 10), parseFloat('2.5kg'));

const strictInteger = (text) =>
  (/^[1-9]\d*$/.test(text.trim()) ? Number(text) : null);

console.log(strictInteger('12'), strictInteger(''), strictInteger('1e3'));
```

Output

```
0 0 1000 16
NaN 12 2.5
12 null null
```

ফাঁকা লেখা `0` হয়ে যায়, `'1e3'` হয় ১০০০, `parseInt` সংখ্যার পরের বাড়তি অংশ নীরবে ফেলে দেয়। পরিমাণের মতো ঘরে এর কোনোটিই গ্রহণযোগ্য নয়। তাই Project আগে লেখার গঠন যাচাই করে, তারপর রূপান্তর করে।

আরেকটি সূক্ষ্ম ফাঁদ: একটি Object-এ Key দিয়ে খোঁজা।

own-keys.js · JavaScript চলবে: Browser Console ও Node.js

```
const catalog = { keyboard: { name: 'Keyboard', price: 1250 } };

const unsafeFind = (id) => catalog[id];
const safeFind = (id) => (Object.hasOwn(catalog, id) ? catalog[id] : null);

console.log(typeof unsafeFind('toString'));
console.log(safeFind('toString'));
console.log(safeFind('keyboard').price);
```

Output

```
function
null
1250
```

প্রতিটি Object উত্তরাধিকারসূত্রে `toString`, `constructor`-এর মতো কিছু Property পায় (অধ্যায় ১৩)। ব্যবহারকারী Form-এর পণ্য-তালিকায় হাত দিয়ে `toString` পাঠালে `catalog[id]` কিছু একটা পায়, এবং Program বিভ্রান্ত হয়। `Object.hasOwn` (অধ্যায় ৭) শুধু Object-এর নিজের Key গ্রহণ করে।

Defensive Program-এর মূল নিয়মগুলো:

- **শুরুতেই যাচাই (Guard Clause),** এবং ভুল হলে সঙ্গে সঙ্গে থামা (Fail Fast)। ভুল তথ্য নিয়ে অনেক দূর এগোলে ত্রুটি দেখা দেয় এমন জায়গায়, যেখানে কারণ আর খুঁজে পাওয়া যায় না।
- **গঠন ও সীমা দুটোই:** ধরন ঠিক কি না, এবং মান গ্রহণযোগ্য সীমার মধ্যে কি না।

- **অনুমোদিত তালিকা (Allow-list):** "এগুলো বাদে সব চলবে" নয়, "শুধু এগুলোই চলবে"।
- **হিসাব পূর্ণসংখ্যায়:** টাকার হিসাব পয়সায় বা পূর্ণ টাকায় রাখুন; দশমিকের হিসাবে `0.1 + 0.2` সমস্যা (অধ্যায় ২)।
- **Browser-এর যাচাই সুবিধা, নিরাপত্তা নয়:** Server-কে সব সময় নিজে আবার যাচাই করতে হবে।

১১.৭ Asynchronous ত্রুটি এবং শেষ প্রতিরক্ষা

`async` Function-এর ত্রুটি `await` এবং `try` / `catch` দিয়ে ধরা হয় (অধ্যায় ১০)। কিন্তু কোথাও `catch` ভুলে গেলে কী হয়? Browser-এ দুটি Global Event শেষ প্রতিরক্ষা হিসেবে কাজ করে: ধরা না পড়া সাধারণ ত্রুটির জন্য `error`, আর ধরা না পড়া Promise-এর ত্রুটির জন্য `unhandledrejection`।

global-handlers.js · JavaScript — চলবে: Browser

```
window.addEventListener('unhandledrejection', (event) => {
  console.log(`ধরা পড়েনি এমন Promise: ${event.reason.message}`);
  event.preventDefault();
});

async function saveDraft() {
  throw new Error('Server বন্ধ');
}

saveDraft();
console.log('saveDraft() ডাকা হয়েছে, কিন্তু await বা catch নেই');
```

Output

```
saveDraft() ডাকা হয়েছে, কিন্তু await বা catch নেই
ধরা পড়েনি এমন Promise: Server বন্ধ
```

এই Handler-গুলো ত্রুটি **ঠিক** করে না; তারা শুধু নিশ্চিত করে যে কোনো ত্রুটি নিঃশব্দে হারিয়ে না যায়। বাস্তব Application-এ এখানে ত্রুটিটি একটি Error Reporting Service-এ পাঠানো হয়, যাতে ব্যবহারকারীর Browser-এ ঘটা Bug-ও ডেভেলপার জানতে পারেন। `event.preventDefault()` Browser-এর নিজস্ব লাল বার্তা বন্ধ করে, কারণ এখানে আমরা নিজেরাই তা সামলেছি। আসল সমাধান অবশ্য একটিই: `saveDraft()`-এর আগে `await` এবং চারপাশে `try` / `catch`।

১১.৮ Stack Trace পড়া এবং DevTools

ধরা না পড়া একটি ত্রুটি Console-এ একটি **Stack Trace** দেখায়: ত্রুটির বার্তা, তারপর কোন Function কোন Function-কে ডেকেছিল তার তালিকা, সবচেয়ে ভেতরেরটি সবার ওপরে।

stack.js · JavaScript চলবে: Node.js

```
function checkStock(product) {
  return product.stock > 0;
}

function addToCart(cart, product) {
  if (checkStock(product)) {
    cart.push(product);
  }
}

addToCart([], undefined);
```

Error Message

```
TypeError: Cannot read properties of undefined (reading 'stock')
at checkStock
at addToCart
```

পড়ার নিয়ম: প্রথম লাইন বলে **কী** ঘটেছে; পরের লাইনগুলো বলে **কোথায়**, প্রতিটির সঙ্গে File-এর নাম ও লাইন নম্বর। ত্রুটি ঘটেছে `checkStock`-এ, কিন্তু ভুলটি সেখানে নয়: `checkStock` ঠিক কাজ করছে। ভুল হলো `addToCart`-কে `undefined` পণ্য দিয়ে ডাকা। তাই Stack-এর ওপর থেকে নিচে নামুন, এবং প্রতিটি স্তরে প্রশ্ন করুন: এই মানটি কোথা থেকে এলো?

DevTools-এর যেসব সুবিধা এ কাজে লাগে:

- **Breakpoint:** Sources Panel-এ লাইন নম্বরে Click করলে Program সেখানে থামে; Scope অংশে সব Variable-এর তখনকার মান দেখা যায়। Code-এ `debugger;` লিখলেও DevTools খোলা থাকলে সেখানে থামে; কাজ শেষে সেটি মুছে ফেলুন।
- **Pause on exceptions:** Sources Panel-এর এই বিকল্প চালু করলে ত্রুটি ঘটার মুহূর্তে Program থামে, Stack ও Variable অক্ষত অবস্থায়।
- **Step Over / Step Into:** থামার পর এক লাইন করে এগোনো, অথবা একটি Function-এর ভেতরে ঢোকা।
- **Conditional Breakpoint:** শুধু একটি শর্ত সত্য হলে থামা, যেমন `product === undefined`; বড় Loop-এ একটি নির্দিষ্ট ঘটনা ধরতে কাজে লাগে।
- **Console:** `console.error` লাল বার্তা ও Stack দেখায়; `console.table(array)` Object-এর তালিকা সারণি আকারে দেখায়।

Debugging-এর পদ্ধতি সব সময় একই: সমস্যাটি নিশ্চিতভাবে আবার ঘটাতে পারা, ধারণা তৈরি করা ("পণ্যটি `undefined`"), প্রমাণ দেখা (Breakpoint বা `console.log`), ঠিক করা, এবং একটি Test যোগ করা যাতে একই Bug আর না ফেরে।

১১.৯ সাধারণ ভুল

ভুল	ফল	সমাধান
খালি `catch {}`	Bug নিঃশব্দে লুকায়	প্রত্যাশিত ধরন যাচাই; বাকিগুলো আবার `throw`
সব কিছু `try`-এর ভেতরে	কোন লাইনে ভুল বোঝা যায় না; অপ্রত্যাশিত ত্রুটিও ধরা পড়ে	`try` শুধু যে লাইন ব্যর্থ হতে পারে তার চারপাশে
লেখা `throw` করা (`throw 'ভুল'`)	`stack` থাকে না, `instanceof Error` মিথ্যা	`throw new Error('ভুল')`
`message`-এর লেখা দেখে সিদ্ধান্ত	সংস্করণ বদলালে Code ভাঙে	`name`, `instanceof` বা নিজের Property
ব্যবহারকারীকে `error.message` বা Stack দেখানো	বিভ্রান্তি; ভেতরের তথ্য ফাঁস	ব্যবহারকারীর ভাষায় বার্তা; বিবরণ Console-এ
মূল ত্রুটি হারিয়ে নতুন ত্রুটি	আসল কারণ খুঁজে পাওয়া যায় না	`{ cause: error }`
পরিষ্কারের কাজ শুধু সফল পথে	ত্রুটির পর Button চিরতরে বন্ধ	`finally`
`await` ছাড়া Promise-এর চারপাশে `try`	ত্রুটি ধরা পড়ে না	`await`
নিজের Class-এ `this.name` না বসানো	Console-এ শুধু `Error`	`this.name = 'ValidationError'`
ত্রুটির পর ভুল অবস্থায় চলতে থাকা	ভুল তথ্য সংরক্ষিত হয়	অপ্রত্যাশিত ত্রুটিতে কাজ থামান

১১.১০ Debugging নির্দেশিকা

একটি ত্রুটির বার্তা পেলে:

১. **প্রথম ত্রুটিটি পড়ুন,** পরেরগুলো প্রায়ই তারই ফল। নাম ও বার্তা দুটোই পড়ুন: `Cannot read properties of undefined (reading 'stock')` মানে যার `stock` পড়া হচ্ছিল, সেটি `undefined`।

২. **Stack-এর প্রথম লাইনে আপনার নিজের File খুঁজুন,** Library-র ভেতরের লাইন নয়।

৩. **সেই লাইনে Breakpoint দিন,** অথবা ঠিক আগে মানটি `console.log` করুন।

৪. **মানটি কোথা থেকে এলো,** Stack ধরে পেছনে যান।

৫. **কোনো ত্রুটির বার্তা না থাকলে** কিন্তু ফল ভুল, তাহলে Logical Error: কয়েকটি জানা ইনপুট দিয়ে প্রত্যাশিত ফল লিখুন এবং কোথায় পার্থক্য শুরু হয় তা খুঁজুন।

৬. **কোনো বার্তা নেই, কিছু ঘটেও না?** একটি খালি `catch` খুঁজুন।

৭. **ঠিক করার পর একটি Test লিখুন,** যা আগের Code-এ ব্যর্থ হতো।

১১.১১ পেশাদার অভ্যাস

- **প্রত্যাশিত ত্রুটি নিজের ধরনে** (`ValidationError` , `ServiceError`); অপ্রত্যাশিত ত্রুটি একেবারে ওপরের এক জায়গায়।
- **`catch` -এ সব সময় ধরুন যাচাই;** অচেনা ত্রুটি আবার `throw` ।
- **ব্যবহারকারীর জন্য বার্তা, ডেভেলপারের জন্য বিবরণ:** দুটি আলাদা জায়গায়।
- **প্রতিটি ব্যবহারকারী-বার্তা বলবে কী করতে হবে:** "সঠিক একটি Email লিখুন", "একটু পরে আবার চেষ্টা করুন"।
- **একসঙ্গে সব ভুল দেখান,** একটি একটি করে নয়; ব্যবহারকারীকে বারবার পাঠাতে বাধ্য করবেন না।
- **`finally` -তে পরিষ্কার।**
- **Trust Boundary-তে একবার পূর্ণ যাচাই;** ভেতরের Code-এ বারবার নয়।
- **ত্রুটির পথেরও Test লিখুন,** শুধু সফল পথের নয়।

১১.১২ Project: OrderGuard — Order Validation

একটি ছোট দোকানের Order Form: গ্রাহকের নাম, Email, পণ্য ও পরিমাণ যাচাই করে Order পাঠানো হয়। লক্ষ্য তিন শ্রেণির ত্রুটি আলাদাভাবে সামলানো: ব্যবহারকারীর ভুল তথ্য (প্রতিটি ঘরের পাশে বার্তা), Server-এর প্রত্যাখ্যান বা Network বিচ্ছিন্নতা (সাধারণ বার্তা, আবার চেষ্টার সুযোগ), এবং Program-এর অপ্রত্যাশিত ভুল (নিরাপদ বার্তা, Console-এ পুরো বিবরণ)।

> **নোট:** এটি একটি শিক্ষামূলক Project। কোনো আসল Server বা টাকা লেনদেন নেই; `createDemoServer` একটি কৃত্রিম Server, যা ৩০০ মিলিসেকেন্ড পরে সাড়া দেয় এবং স্টক হিসাব রাখে। Order-এর ইতিহাস শুধু Page খোলা থাকা পর্যন্ত থাকে।

Project-এর চাহিদা

- নাম ২ থেকে ৬০ অক্ষর; Email-এর মোটামুটি গঠন; পণ্য শুধু তালিকা থেকে; পরিমাণ ১ থেকে ১০০-এর পূর্ণসংখ্যা।
- সব ভুল একসঙ্গে, প্রতিটি ঘরের নিচে; প্রথম ভুল ঘরে Focus; `aria-invalid` ও `aria-describedby` ।
- পাঠানোর সময় Button বন্ধ এবং "পাঠানো হচ্ছে…"; সফল, ব্যর্থ বা অপ্রত্যাশিত—সব ক্ষেত্রে পরে Button আবার চালু।
- স্টকে যথেষ্ট পণ্য না থাকলে Server-এর বার্তা; Network বিচ্ছিন্ন হলে আবার চেষ্টার বার্তা।
- সফল হলে রসিদ নম্বর, টাকার অঙ্কে মোট দাম এবং ইতিহাসে একটি সারি।
- সব লেখা `textContent` দিয়ে।

File-এর কাঠামো

Folder-এর কাঠামো

```
order-validation/
├── index.html
├── style.css
├── order-validation.js        যাচাই, Error-এর ধরন, কৃত্রিম Server
├── app.js                     Form, বার্তা ও ইতিহাস
├── README.md
└── tests/
    ├── load-script.js
    ├── order-validation.test.js
    ├── cdp.mjs
    └── browser-check.mjs
```

order-validation.js

দুই অংশ মিলে সম্পূর্ণ File।

order-validation.js · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Browser Console ও Node.js

```
'use strict';

const CATALOG = {
  keyboard: { name: 'Keyboard', price: 1250 },
  mouse: { name: 'Mouse', price: 650 },
  headset: { name: 'Headset', price: 2400 },
};
const STOCK = { keyboard: 10, mouse: 5, headset: 2 };
const MAX_QUANTITY = 100;

// প্রত্যাশিত: ব্যবহারকারী ঠিক করতে পারেন। fields-এ কোন ঘরে কী সমস্যা।
class ValidationError extends Error {
  constructor(fields) {
    super('কয়েকটি ঘর ঠিক করতে হবে।');
    this.name = 'ValidationError';
    this.fields = fields;
  }
}

// প্রত্যাশিত: Server অনুরোধটি গ্রহণ করেনি (যেমন স্টক শেষ)।
class ServiceError extends Error {
  constructor(message) {
    super(message);
    this.name = 'ServiceError';
  }
}

function parseQuantity(value) {
```

```
  const text = String(value ?? '').trim();

  if (!/^[1-9]\d*$/.test(text)) {
    return null;
  }
  const quantity = Number(text);
  return quantity <= MAX_QUANTITY ? quantity : null;
}

function formatTaka(amount) {
  if (!Number.isSafeInteger(amount) || amount < 0) {
    throw new RangeError(`অবৈধ টাকার অঙ্ক: ${amount}`);
  }
  return `${amount.toLocaleString('en-US')} টাকা`;
}

// ...
```

order-validation.js · JavaScript (অংশ ২ / ২ (Part 2 of 2)) চলবে: Browser Console ও Node.js

```
// ...

// Trust Boundary: Form-এর তথ্য যাচাই করে পরিষ্কার Order, নয়তো ValidationError।
function validateOrder(raw, catalog = CATALOG) {
  if (raw === null || typeof raw !== 'object') {
    throw new TypeError('validateOrder-কে একটি Object দিতে হবে');
  }

  const customer = String(raw.customer ?? '').trim();
  const email = String(raw.email ?? '').trim();
  const productId = String(raw.productId ?? '');
  const product = Object.hasOwn(catalog, productId) ? catalog[productId] : null;
  const quantity = parseQuantity(raw.quantity);
  const fields = {};

  if (customer.length < 2 || customer.length > 60) {
    fields.customer = 'নাম 2 থেকে 60 অক্ষরের হতে হবে।';
  }
  if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email)) {
    fields.email = 'সঠিক একটি Email লিখুন।';
  }
  if (product === null) {
    fields.productId = 'তালিকা থেকে একটি পণ্য বাছুন।';
  }
  if (quantity === null) {
    fields.quantity = `পরিমাণ 1 থেকে ${MAX_QUANTITY}-এর মধ্যে পূর্ণসংখ্যা হতে হবে।`;
  }
  if (Object.keys(fields).length > 0) {
    throw new ValidationError(fields);
```

```
  }

  const total = product.price * quantity;
  const productName = product.name;
  return { customer, email, productId, productName, quantity, total };
}

// কৃত্রিম Server: অপেক্ষার পর রসিদ নম্বর, অথবা স্টক না থাকলে ServiceError।
function createDemoServer(stock = STOCK, delayMs = 300) {
  const left = { ...stock };
  let next = 1;

  return async function send(order) {
    await new Promise((resolve) => setTimeout(resolve, delayMs));
    const available = left[order.productId] ?? 0;

    if (order.quantity > available) {
      const name = order.productName;
      throw new ServiceError(`দুঃখিত, ${name} স্টকে আছে ${available}টি।`);
    }
    left[order.productId] = available - order.quantity;
    return `ORD-${String(next++).padStart(4, '0')}`;
  };
}

// প্রত্যাশিত ত্রুটি ফলাফল হিসেবে ফেরত; অপ্রত্যাশিত ত্রুটি ওপরের স্তরে যায়।
async function placeOrder(raw, send) {
  let order;
  try {
    order = validateOrder(raw);
  } catch (error) {
    if (error instanceof ValidationError) {
      const { message, fields } = error;
      return { ok: false, kind: 'validation', message, fields };
    }
    throw error;
  }

  try {
    return { ok: true, order, receipt: await send(order) };
  } catch (error) {
    if (error instanceof ServiceError) {
      return { ok: false, kind: 'service', message: error.message };
    }
    const message =
      'Server-এর সঙ্গে যোগাযোগ করা যায়নি। একটু পরে আবার চেষ্টা করুন।';
    return { ok: false, kind: 'network', message, cause: error };
  }
}
```

- **দুটি Error-এর ধরন, দুটিই প্রত্যাশিত।** `ValidationError` বহন করে প্রতিটি ঘরের বার্তা; `ServiceError` বহন করে Server-এর ব্যাখ্যা।
- `validateOrder` **সব ঘর যাচাই করে, তারপর একবারে** `throw`**।** প্রথম ভুলেই থামলে ব্যবহারকারীকে একটি করে ভুল ঠিক করে বারবার পাঠাতে হতো।
- `validateOrder`**-এর প্রথম** `throw` **ভিন্ন জাতের।** `raw` একটি Object না হওয়া ব্যবহারকারীর ভুল নয়; এর মানে Code ভুলভাবে Function-টি ডেকেছে। তাই সেটি `TypeError`, এবং `placeOrder` তা ধরে না, ওপরে পাঠিয়ে দেয়।
- `placeOrder`**-এর দুটি আলাদা** `try`**।** প্রথমটি যাচাইয়ের, দ্বিতীয়টি পাঠানোর। একটি বড় `try` হলে যাচাইয়ের একটি Bug ভুল করে "Network বিচ্ছিন্ন" বার্তা দেখাত।
- **প্রত্যাশিত ত্রুটি ফলাফল হয়ে ফেরে** (`{ ok: false, kind, message }`), তাই Page-এর Code-কে প্রতিটি ধরন চিনতে হয় না। Network-এর আসল ত্রুটি `cause`-এ থাকে, ডেভেলপারের জন্য।
- `formatTaka` **নিজেও যাচাই করে।** অবৈধ অঙ্ক এলে তা Program-এর ভুল, তাই `RangeError`।

index.html ও style.css

index.html · HTML অংশ ১ / ২ (Part 1 of 2) চলবে: Browser

```
<!doctype html>
<html lang="bn">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>OrderGuard</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <main class="app">
    <h1>OrderGuard</h1>

    <form id="order-form" novalidate>
      <label for="customer">গ্রাহকের নাম</label>
      <input id="customer" name="customer" autocomplete="name"
             aria-describedby="customer-error">
      <p id="customer-error" class="field-error"></p>

      <label for="email">Email</label>
      <input id="email" name="email" type="email" autocomplete="email"
             aria-describedby="email-error">
      <p id="email-error" class="field-error"></p>
<!-- ... -->
```

index.html · HTML অংশ ২ / ২ (Part 2 of 2) চলবে: Browser

```
<!-- ... -->
      <label for="productId">পণ্য</label>
      <select id="productId" name="productId" aria-describedby="productId-error">
```

```
        <option value="">বাছুন</option>
        <option value="keyboard">Keyboard (1,250 টাকা)</option>
        <option value="mouse">Mouse (650 টাকা)</option>
        <option value="headset">Headset (2,400 টাকা)</option>
      </select>
      <p id="productId-error" class="field-error"></p>

      <label for="quantity">পরিমাণ</label>
      <input id="quantity" name="quantity" inputmode="numeric"
             aria-describedby="quantity-error">
      <p id="quantity-error" class="field-error"></p>

      <button type="submit">Order পাঠান</button>
    </form>

    <p id="status" role="status" aria-live="polite"></p>

    <h2>গৃহীত Order</h2>
    <ul id="history"></ul>
  </main>

  <script src="order-validation.js"></script>
  <script src="app.js"></script>
</body>
</html>
```

style.css · CSS — চলবে: Browser

```
body {
  margin: 0;
  font-family: 'Hind Siliguri', 'Nirmala UI', sans-serif;
  background: #f4f6fa;
  color: #1c2430;
}

.app {
  max-width: 560px;
  margin: 32px auto;
  padding: 24px;
  background: #ffffff;
  border-radius: 12px;
}

#order-form {
  display: grid;
  gap: 4px;
}

input,
```

```
select,
button {
  font: inherit;
  padding: 6px 10px;
}

[aria-invalid='true'] {
  border: 2px solid #b3261e;
}

.field-error {
  min-height: 1.2em;
  margin: 0 0 8px;
  color: #b3261e;
}
```

ভুল ঘরের লাল সীমানা নির্ধারিত হয় `aria-invalid` দিয়ে, তাই চোখে দেখা চিহ্ন এবং Screen Reader-এর তথ্য সব সময় একসঙ্গে বদলায়।

app.js

দুই অংশ মিলে সম্পূর্ণ File।

app.js · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Browser

```
'use strict';

// ?offline=1 দিলে Network বিচ্ছিন্নতার অভিনয়; ?delay= কৃত্রিম Server-এর দেরি।
const params = new URLSearchParams(location.search);
const send = params.has('offline')
  ? async () => { throw new TypeError('Failed to fetch'); }
  : createDemoServer(STOCK, Number(params.get('delay') ?? 300));

const FIELDS = ['customer', 'email', 'productId', 'quantity'];
const form = document.querySelector('#order-form');
const submitButton = form.querySelector('button[type="submit"]');
const statusView = document.querySelector('#status');
const historyList = document.querySelector('#history');

function showFieldErrors(fields = {}) {
  for (const name of FIELDS) {
    const message = fields[name] ?? '';
    document.querySelector(`#${name}-error`).textContent = message;
    form.elements[name].setAttribute('aria-invalid', String(message !== ''));
  }

  const firstInvalid = FIELDS.find((name) => fields[name]);
  if (firstInvalid) {
    form.elements[firstInvalid].focus();
```

```
  }
}

function addToHistory(order, receipt) {
  const item = document.createElement('li');
  item.textContent =
    `${receipt}: ${order.customer}, ${order.productName} ${order.quantity}টি, ` +
    formatTaka(order.total);
  historyList.prepend(item);
}

// ...
```

app.js · JavaScript অংশ ২ / ২ (Part 2 of 2) চলবে: Browser

```
// ...

form.addEventListener('submit', async (event) => {
  event.preventDefault();
  submitButton.disabled = true;
  statusView.textContent = 'পাঠানো হচ্ছে...';

  try {
    const result = await placeOrder(Object.fromEntries(new FormData(form)), send);

    if (result.ok) {
      showFieldErrors();
      addToHistory(result.order, result.receipt);
      statusView.textContent =
        `Order গৃহীত: ${result.receipt}, মোট ${formatTaka(result.order.total)}।`;
      form.reset();
      return;
    }

    showFieldErrors(result.fields);
    statusView.textContent = result.message;
    if (result.cause) {
      console.warn('Order পাঠানো যায়নি:', result.cause);
    }
  } catch (error) {
    // অপ্রত্যাশিত: Program-এর ভুল। ব্যবহারকারী পান নিরাপদ বার্তা,
    // ডেভেলপার পান পুরো বিবরণ।
    statusView.textContent =
      'অপ্রত্যাশিত সমস্যা হয়েছে। Page Reload করে আবার চেষ্টা করুন।';
    console.error('OrderGuard-এ অপ্রত্যাশিত ত্রুটি:', error);
  } finally {
    submitButton.disabled = false;
  }
});
```

Code-এর ধাপে ধাপে ব্যাখ্যা

- **Button বন্ধ করা প্রথমেই,** যাতে পাঠানোর সময় দুবার Click-এ দুটি Order না যায়। আবার চালু হয় `finally`-তে: সফল হলে `return`-এর পরেও, প্রত্যাশিত ব্যর্থতায়, এমনকি অপ্রত্যাশিত ত্রুটিতেও।
- **`Object.fromEntries(new FormData(form))`** Form-এর সব ঘর একবারে একটি Object-এ আনে: Key হলো প্রতিটি ঘরের `name`। Trust Boundary-র ভেতরে ঢোকার আগে এটিই কাঁচা তথ্য।
- **তিনটি পথ।** সফল: ঘরের বার্তা মুছে ইতিহাস ও রসিদ। প্রত্যাশিত ব্যর্থতা: `result.fields` থাকলে প্রতিটি ঘরের নিচে বার্তা, এবং ওপরে সাধারণ বার্তা। অপ্রত্যাশিত: `catch`-এ নিরাপদ বার্তা এবং `console.error`।
- **`showFieldErrors()` Argument ছাড়া** ডাকলে Default Parameter `{}` সব বার্তা মুছে দেয়, এবং সব ঘর `aria-invalid="false"` হয়।
- **`console.warn` বনাম `console.error`।** Network বিচ্ছিন্নতা প্রত্যাশিত, তাই সতর্কতা; শুধু অপ্রত্যাশিত ত্রুটি লাল `error`। Console দেখে এক নজরে বোঝা যায় কোনটি Bug।
- **কৃত্রিম Network বিচ্ছিন্নতা।** `?offline=1` দিলে `send` সব সময় `TypeError` ছোড়ে, ঠিক যেমন বিচ্ছিন্ন অবস্থায় `fetch` করে (অধ্যায় ১০)।

চালানো এবং প্রত্যাশিত আচরণ

`index.html` Browser-এ খুলুন (Double Click)। Test চালাতে Project Folder থেকে:

Terminal

```
node --test
node tests/browser-check.mjs
```

ইনপুট	যা হওয়ার কথা
সব ঘর ফাঁকা	চারটি ঘরের নিচেই বার্তা; নামের ঘরে Focus
Sara, `sara@example.com`, Keyboard, 2	"Order গৃহীত: ORD-0001, মোট 2,500 টাকা।"; ইতিহাসে সারি; Form খালি
পরিমাণ `0`, `2.5`, `1e3` বা `101`	পরিমাণের ঘরে বার্তা
Headset, পরিমাণ 3	"দুঃখিত, Headset স্টকে আছে 2টি।"
`index.html?offline=1` থেকে বৈধ Order	Network-এর বার্তা; Button আবার চালু
নামে `<img src="" onerror="...">`	লেখাটি হুবহু ইতিহাসে; কোনো Code চলে না

Companion Source Code-এর `order-validation.test.js` Node.js-এ ১৬টি Test চালায়, এবং `browser-check.mjs` আসল Chrome-এ Page খুলে ওপরের সব পরিস্থিতিসহ ১২টি আচরণ যাচাই করে, যার মধ্যে Button-এর বন্ধ ও চালু হওয়া এবং ইচ্ছা করে ঢোকানো একটি Bug-এর অপ্রত্যাশিত ত্রুটির পথও আছে।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
ত্রুটির পর Button আর চালু হয় না	চালু করার Code `finally`-র বাইরে	`finally`
সব ভুলে একই "Network" বার্তা	যাচাই ও পাঠানো একই `try`-তে	দুটি আলাদা `try`
Bug-এর ত্রুটি Console-এ দেখা যায় না	খালি বা অতি সাধারণ `catch`	ধরন যাচাই, অচেনা হলে `throw`
ঘরের লাল সীমানা থেকে যায়	`aria-invalid` মোছা হয়নি	সফল হলে `showFieldErrors()`
পণ্যের ঘরে `toString` পাঠালে অদ্ভুত আচরণ	`catalog[id]` সরাসরি	`Object.hasOwn`
`1e3` বা `2.5` পরিমাণ গৃহীত	`Number()` দিয়ে সরাসরি রূপান্তর	`parseQuantity`-এর গঠন যাচাই

এই Project-এর সীমাবদ্ধতা

এটি একটি শিক্ষামূলক Project। Server কৃত্রিম, তাই স্টক Page Reload করলে আবার শুরু থেকে গোনে। Email যাচাই শুধু মোটা দাগের গঠনের (অধ্যায় ৮)। আসল দোকানে একই যাচাই Server-এও হতো, কারণ Browser-এর Code ব্যবহারকারীর নিয়ন্ত্রণে।

১১.১৩ অনুশীলনী

প্রাথমিক স্তর

১. নিচের প্রতিটি কোন ধরনের ত্রুটি দেয়, না চালিয়ে বলুন: `undefined.x`, `missingName`, `new Array(-1)`, `JSON.parse('')`। তারপর Code দিয়ে যাচাই করুন।

২. `safeDivide(a, b)` লিখুন, যা `b` শূন্য হলে `RangeError` ছোড়ে। `try` / `catch` দিয়ে দুটি ক্ষেত্র দেখান।

৩. একটি Function-এর `try`-তে `return 'ক'` এবং `finally`-তে `console.log('খ')` আছে। Function ডেকে ফল ছাপলে কোন ক্রমে কী দেখা যাবে?

৪. `try { saveOrder(); } catch (error) {}` Code-টির সমস্যা কী? উন্নত করুন।

৫. `requireText(value, field)` লিখুন: `value` লেখা না হলে `TypeError`, ফাঁকা হলে `ValidationError`।

মধ্যম স্তর

৬. `parseJsonOr(text, fallback)` লিখুন: ভাঙা JSON-এ `fallback` ফেরত দেয়, কিন্তু অন্য যেকোনো ত্রুটি আবার ছোড়ে।

৭. `NotFoundError` তৈরি করুন, যাতে খোঁজা `id` থাকে; একটি `findProduct` Function-এ ব্যবহার করুন।

৮. `readConfig(text)` লিখুন, যা ভাঙা JSON-এ `'Config পড়া যায়নি'` বার্তার নতুন Error ছোড়ে এবং মূল ত্রুটি `cause`-এ রাখে।

৯. একটি `async` Function `loadProfile` লিখুন যা কৃত্রিমভাবে ব্যর্থ হয়, এবং `try` / `catch` / `finally` দিয়ে ত্রুটির বার্তা ও "Loading বন্ধ" দেখান।

চ্যালেঞ্জ স্তর

১০. `validateSignup(form)` লিখুন, যা নাম, Email ও Password-এর সব ভুল একসঙ্গে একটি Object-এ জমা করে `ValidationError` ছোড়ে।

১১. `retryIfAllowed(task, times)` লিখুন, যা শুধু `error.retryable === true` ত্রুটিতে আবার চেষ্টা করে।

১২. `attempt(fn)` লিখুন, যা `{ ok: true, value }` বা `{ ok: false, error }` ফেরত দেয়, কখনো ছোড়ে না। কখন এই ধরন `throw`-এর চেয়ে ভালো?

১১.১৪ অনুশীলনীর সমাধান

সমাধান ১

which-error.js · JavaScript — চলবে: Browser Console ও Node.js

```
const cases = [
  () => undefined.x,
  () => missingName,
  () => new Array(-1),
  () => JSON.parse(''),
];

for (const run of cases) {
  try {
    run();
  } catch (error) {
    console.log(error.name);
  }
}
```

Output

```
TypeError
ReferenceError
RangeError
SyntaxError
```

সমাধান ২

safe-divide.js · JavaScript — চলবে: Browser Console ও Node.js

```js
function safeDivide(a, b) {
  if (b === 0) {
    throw new RangeError('শূন্য দিয়ে ভাগ করা যায় না');
  }
  return a / b;
}

for (const [a, b] of [[10, 4], [10, 0]]) {
  try {
    console.log(safeDivide(a, b));
  } catch (error) {
    console.log(`${error.name}: ${error.message}`);
  }
}
```

Output

```
2.5
RangeError: শূন্য দিয়ে ভাগ করা যায় না
```

JavaScript নিজে `10 / 0`-এ ত্রুটি দেয় না, `Infinity` দেয়; তাই নিয়মটি নিজে যাচাই করতে হয়েছে।

সমাধান ৩

finally-order.js · JavaScript — চলবে: Browser Console ও Node.js

```js
function demo() {
  try {
    return 'ক';
  } finally {
    console.log('খ');
  }
}

console.log(demo());
```

Output

```
খ
ক
```

ফেরত মান তৈরি হয়েছে, কিন্তু Function ফিরে যাওয়ার আগে `finally` চলেছে; তারপর ডাকার জায়গায় `'ক'` ছাপা হয়েছে।

সমাধান ৪ — খালি `catch` সব ত্রুটি চাপা দেয়: Order আদৌ সংরক্ষিত হলো কি না, কেউ জানতে পারেন না, এবং Code-এর Bug-ও লুকিয়ে যায়। প্রত্যাশিত ত্রুটি চিনে ব্যবহারকারীকে জানান, বাকিগুলো আবার ছুড়ুন:

no-empty-catch.js · JavaScript — চলবে: Browser Console ও Node.js

```javascript
class ServiceError extends Error {}

function saveOrder() {
  throw new ServiceError('স্টক শেষ');
}

try {
  saveOrder();
} catch (error) {
  if (!(error instanceof ServiceError)) {
    throw error;
  }
  console.log(`Order সংরক্ষণ হয়নি: ${error.message}`);
}
```

Output

```
Order সংরক্ষণ হয়নি: স্টক শেষ
```

সমাধান ৫

require-text.js · JavaScript — চলবে: Browser Console ও Node.js

```javascript
class ValidationError extends Error {
  constructor(message) {
    super(message);
    this.name = 'ValidationError';
  }
}

function requireText(value, field) {
  if (typeof value !== 'string') {
    throw new TypeError(`${field}: লেখা আশা করা হয়েছিল`);
```

```
  }
  if (value.trim() === '') {
    throw new ValidationError(`${field} ঘরটি পূরণ করুন`);
  }
  return value.trim();
}

for (const value of [' Sara ', '   ', 42]) {
  try {
    console.log(`[${requireText(value, 'নাম')}]`);
  } catch (error) {
    console.log(`${error.name}: ${error.message}`);
  }
}
```

Output

```
[Sara]
ValidationError: নাম ঘরটি পূরণ করুন
TypeError: নাম: লেখা আশা করা হয়েছিল
```

ফাঁকা ঘর ব্যবহারকারীর ভুল (`ValidationError`); সংখ্যা আসা মানে Code ভুলভাবে ডেকেছে (`TypeError`)।

সমাধান ৬

parse-json-or.js · JavaScript — চলবে: Browser Console ও Node.js

```
function parseJsonOr(text, fallback) {
  try {
    return JSON.parse(text);
  } catch (error) {
    if (error instanceof SyntaxError) {
      return fallback;
    }
    throw error;
  }
}

console.log(parseJsonOr('{"theme":"dark"}', {}).theme);
console.log(parseJsonOr('{broken', { theme: 'light' }).theme);
```

Output

```
dark
light
```

সমাধান ৭

not-found.js · JavaScript — চলবে: Browser Console ও Node.js

```
class NotFoundError extends Error {
  constructor(id) {
    super(`পণ্য পাওয়া যায়নি: ${id}`);
    this.name = 'NotFoundError';
    this.id = id;
  }
}

const products = [{ id: 'P1', name: 'Keyboard' }];

function findProduct(id) {
  const product = products.find((item) => item.id === id);
  if (!product) {
    throw new NotFoundError(id);
  }
  return product;
}

try {
  console.log(findProduct('P1').name);
  findProduct('P9');
} catch (error) {
  console.log(error instanceof NotFoundError, error.id, error.message);
}
```

Output

```
Keyboard
true P9 পণ্য পাওয়া যায়নি: P9
```

সমাধান ৮

read-config.js · JavaScript — চলবে: Browser Console ও Node.js

```
function readConfig(text) {
  try {
    return JSON.parse(text);
  } catch (error) {
    throw new Error('Config পড়া যায়নি', { cause: error });
  }
}

try {
  readConfig('port=8080');
} catch (error) {
  console.log(error.message);
  console.log(error.cause instanceof SyntaxError);
}
```

Output

```
Config পড়া যায়নি
true
```

সমাধান ৯

load-profile.js · JavaScript — চলবে: Browser Console ও Node.js

```
const wait = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

async function loadProfile() {
  await wait(50);
  throw new Error('Profile Server সাড়া দেয়নি');
}

async function main() {
  console.log('Loading শুরু');
  try {
    await loadProfile();
  } catch (error) {
    console.log(`ব্যর্থ: ${error.message}`);
  } finally {
    console.log('Loading বন্ধ');
  }
}

main();
```

Output

```
Loading শুরু
ব্যর্থ: Profile Server সাড়া দেয়নি
Loading বন্ধ
```

সমাধান ১০

validate-signup.js · JavaScript — চলবে: Browser Console ও Node.js

```
class ValidationError extends Error {
  constructor(fields) {
    super('কয়েকটি ঘর ঠিক করতে হবে');
    this.name = 'ValidationError';
    this.fields = fields;
  }
}

function validateSignup({ name = '', email = '', password = '' }) {
  const fields = {};

  if (name.trim().length < 2) {
    fields.name = 'নাম কমপক্ষে 2 অক্ষর';
  }
  if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email)) {
    fields.email = 'সঠিক Email';
  }
  if (password.length < 8) {
    fields.password = 'Password কমপক্ষে 8 অক্ষর';
  }
  if (Object.keys(fields).length > 0) {
    throw new ValidationError(fields);
  }
  return { name: name.trim(), email };
}

try {
  validateSignup({ name: 'S', email: 'sara@', password: '123' });
} catch (error) {
  for (const [field, message] of Object.entries(error.fields)) {
    console.log(`${field}: ${message}`);
  }
}
```

Output

```
name: নাম কমপক্ষে 2 অক্ষর
email: সঠিক Email
password: Password কমপক্ষে 8 অক্ষর
```

সমাধান ১১

retry-if-allowed.js · JavaScript — চলবে: Browser Console ও Node.js

```
async function retryIfAllowed(task, times = 3) {
  for (let attempt = 1; ; attempt += 1) {
    try {
      return await task(attempt);
    } catch (error) {
      if (error.retryable !== true || attempt >= times) {
        throw error;
      }
      console.log(`চেষ্টা ${attempt} ব্যর্থ, আবার`);
    }
  }
}

const busyTwice = async (attempt) => {
  if (attempt < 3) {
    throw Object.assign(new Error('ব্যস্ত'), { retryable: true });
  }
  return 'সফল';
};

const forbidden = async () => {
  throw Object.assign(new Error('অনুমতি নেই'), { retryable: false });
};

async function main() {
  console.log(await retryIfAllowed(busyTwice));
  await retryIfAllowed(forbidden).catch((error) => console.log(error.message));
}

main();
```

Output

```
চেষ্টা 1 ব্যর্থ, আবার
চেষ্টা 2 ব্যর্থ, আবার
সফল
অনুমতি নেই
```

সমাধান ১২

attempt.js · JavaScript চলবে: Browser Console ও Node.js

```
function attempt(fn) {
  try {
    return { ok: true, value: fn() };
  } catch (error) {
    return { ok: false, error };
  }
}

const good = attempt(() => JSON.parse('[1, 2]'));
const bad = attempt(() => JSON.parse('[1, 2'));

console.log(good.ok, good.value.length);
console.log(bad.ok, bad.error.name);
```

Output

```
true 2
false SyntaxError
```

এই ধরন ভালো যেখানে ব্যর্থতা স্বাভাবিক ও ঘনঘন, এবং ডাকার জায়গাতেই তা সামলাতে হবে: যেমন অনেকগুলো সারি পড়া, যার কিছু ভাঙা হতে পারে। Project-এর `placeOrder` প্রত্যাশিত ত্রুটির জন্য এটিই ব্যবহার করেছে। কিন্তু অপ্রত্যাশিত ত্রুটির জন্য `throw` ভালো: সেগুলো নিজে থেকেই ওপরে উঠে যায়, এবং কেউ ভুলে "যাচাই না করা" ফল ব্যবহার করে ফেলতে পারেন না।

১১.১৫ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. কোন ত্রুটি Program চলা শুরুর আগেই ধরা পড়ে?

ক) TypeError

খ) Logical Error

গ) SyntaxError

ঘ) RangeError

প্রশ্ন ২. `null.length` কোন ধরনের ত্রুটি দেয়?

ক) TypeError

খ) ReferenceError

গ) RangeError

ঘ) SyntaxError

প্রশ্ন ৩. `finally` কখন চলে?

ক) শুধু ত্রুটি হলে

খ) শুধু সফল হলে

গ) `try`-তে `return` না থাকলে

ঘ) সব ক্ষেত্রে, `return` থাকলেও

প্রশ্ন ৪. কেন `throw 'ভুল'`-এর বদলে `throw new Error('ভুল')`?

ক) Error Object-এ `stack` এবং `name` থাকে

খ) লেখা `throw` করা যায় না

গ) দ্রুত চলে

ঘ) `catch` শুধু Error ধরে

প্রশ্ন ৫. একটি `catch`-এ অচেনা ধরনের ত্রুটি এলে সাধারণত কী করা উচিত?

ক) চুপচাপ উপেক্ষা

খ) আবার `throw` করা

গ) `return null`

ঘ) Page Reload

প্রশ্ন ৬. কোনটি প্রত্যাশিত ত্রুটি?

ক) নামের বানান ভুলের কারণে `ReferenceError`

খ) `undefined`-এর Property পড়া

গ) ব্যবহারকারী পরিমাণে `0` লিখেছেন

ঘ) ভুল সূত্রে হিসাব

প্রশ্ন ৭. `new Error('Settings পড়া যায়নি', { cause: error })`-এ `cause` কী কাজ করে?

ক) নতুন ত্রুটি চাপা দেয়

খ) মূল ত্রুটি ধরে রাখে, যাতে আসল কারণ খুঁজে পাওয়া যায়

গ) ত্রুটিকে প্রত্যাশিত করে

ঘ) `stack` মুছে দেয়

প্রশ্ন ৮. `Number('')`-এর মান কত?

ক) `NaN`

খ) `0`

গ) `null`

ঘ) `undefined`

> **প্রশ্ন ৯.** `catalog[id]`-এর বদলে `Object.hasOwn(catalog, id)` যাচাই কেন?
>
> ক) দ্রুত চলে
>
> খ) `toString`-এর মতো উত্তরাধিকারসূত্রে পাওয়া Key গ্রহণ না করতে
>
> গ) `id` সংখ্যা হলে কাজ করতে
>
> ঘ) Catalog বদলাতে

> **প্রশ্ন ১০.** Browser-এ ধরা না পড়া Promise-এর ত্রুটি কোন Event-এ জানা যায়?
>
> ক) `error`
>
> খ) `rejected`
>
> গ) `catch`
>
> ঘ) `unhandledrejection`

১১.১৬ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–গ · ২–ক · ৩–ঘ · ৪–ক · ৫–খ · ৬–গ · ৭–খ · ৮–খ · ৯–খ · ১০–ঘ

প্রশ্ন	উত্তর	ব্যাখ্যা
১	গ	JavaScript পুরো File আগে পড়ে; ব্যাকরণের ভুলে কিছুই চলে না।
২	ক	ভুল ধরনের মান (`null`) থেকে Property পড়া।
৩	ঘ	সফল, ব্যর্থ বা আগেভাগে `return`—সব পথে `finally`।
৪	ক	Stack থেকে ত্রুটির জায়গা পাওয়া যায়, `name` ও `instanceof` দিয়ে ধরন চেনা যায়।
৫	খ	না চিনে সামলালে Bug লুকায়; ওপরের স্তর বা Console দেখুক।
৬	গ	ব্যবহারকারী নিজে ঠিক করতে পারেন; বাকিগুলো Code-এর ভুল।
৭	খ	ওপরের স্তরের বার্তার সঙ্গে নিচের স্তরের আসল ত্রুটিও থাকে।
৮	খ	ফাঁকা লেখা `0`-তে রূপান্তরিত হয়, যা যাচাইয়ের একটি ফাঁদ।
৯	খ	প্রতিটি Object উত্তরাধিকারসূত্রে কিছু Property পায়।
১০	ঘ	`error` সাধারণ ত্রুটির জন্য; Promise-এর জন্য `unhandledrejection`।

১১.১৭ অধ্যায়ের সারসংক্ষেপ

- Syntax Error-এ Program চলেই না; Runtime Error চলার সময় থামায়; Logical Error নিঃশব্দে ভুল ফল দেয়, যা Test ধরে।

- Error Object-এ `name`, `message`, `stack`; সিদ্ধান্ত নিন `name` বা `instanceof` দিয়ে, `message`-এর লেখা দিয়ে নয়।
- `throw new Error(...)`; `catch` কাছের ত্রুটি ধরে; `finally` সব পথে চলে, তাই পরিষ্কারের কাজ সেখানে।
- নিজের Error-এর ধরন `class ... extends Error` দিয়ে, `this.name` বসিয়ে; মূল কারণ `cause`-এ।
- প্রত্যাশিত ত্রুটি কাছেই ধরে ব্যবহারকারীকে কী করতে হবে তা জানান; অপ্রত্যাশিত ত্রুটি ওপরের এক জায়গায়, নিরাপদ বার্তা ও পুরো Log।
- `catch`-এ ধরন যাচাই; অচেনা হলে আবার `throw`; খালি `catch` নয়।
- Trust Boundary-তে কঠোর যাচাই: গঠন, সীমা, অনুমোদিত তালিকা, `Object.hasOwn`।
- Asynchronous ত্রুটিতে `await` ও `try`; শেষ প্রতিরক্ষা `error` ও `unhandledrejection`।
- Stack Trace ওপর থেকে পড়ুন, তারপর মান কোথা থেকে এলো তা পেছনে খুঁজুন; Breakpoint ও Pause on exceptions।

পরের অধ্যায়ে Modern JavaScript ও Module: Code-কে আলাদা File-এ ভাগ করা, `import` ও `export`, এবং পরিষ্কার Code লেখার অভ্যাস। এখন পর্যন্ত Project-গুলোতে যে Script-এর ক্রম মেনে চলতে হয়েছে, Module সেই সমস্যার সমাধান করে।

১১.১৮ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
Syntax Error	ব্যাকরণের ভুল; Program চলে না।
Runtime Error	চলার সময় ঘটা ত্রুটি।
Logical Error	Program চলে, কিন্তু ফল ভুল।
`throw`	ত্রুটি ছোড়া; স্বাভাবিক প্রবাহ থামে।
`finally`	সফল বা ব্যর্থ, সব ক্ষেত্রে চলা অংশ।
Custom Error	নিজের তৈরি Error-এর ধরন।
`cause`	নতুন ত্রুটির ভেতরে রাখা মূল ত্রুটি।
Expected Error	পরিবেশ বা ব্যবহারকারীর কারণে, যা স্বাভাবিকভাবে সামলানো যায়।
Unexpected Error	Program-এর ভুল; ডেভেলপারকে ঠিক করতে হয়।
Trust Boundary	যেখানে বাইরের তথ্য Program-এ ঢোকে।
Stack Trace	ত্রুটির মুহূর্তে Function-ডাকার ধারাবাহিক তালিকা।
Breakpoint	DevTools-এ Program থামানোর জায়গা।

১১.১৯ আরও পড়ুন

- **MDN — Control flow and error handling:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Control_flow_and_error_handling` — `try`, `catch`, `throw` ও `finally`।
- **MDN — Error:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Error` — Error-এর ধরন, `cause` এবং নিজের Error।
- **MDN — Window: unhandledrejection event:** `https://developer.mozilla.org/en-US/docs/Web/API/Window/unhandledrejection_event` — ধরা না পড়া Promise-এর ত্রুটি।
- **Chrome DevTools — Debug JavaScript:** `https://developer.chrome.com/docs/devtools/javascript` — Breakpoint ও Step-এর ব্যবহার।
- **OWASP — Input Validation Cheat Sheet:** `https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html` — Trust Boundary-তে যাচাইয়ের নীতি।